



中山大学 软件工程学院
SUN YAT-SEN UNIVERSITY SCHOOL OF SOFTWARE ENGINEERING

SSE206 《计算机网络》

第三章 传输层

南雨宏

nanyh@mail.sysu.edu.cn

2026-04-08

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

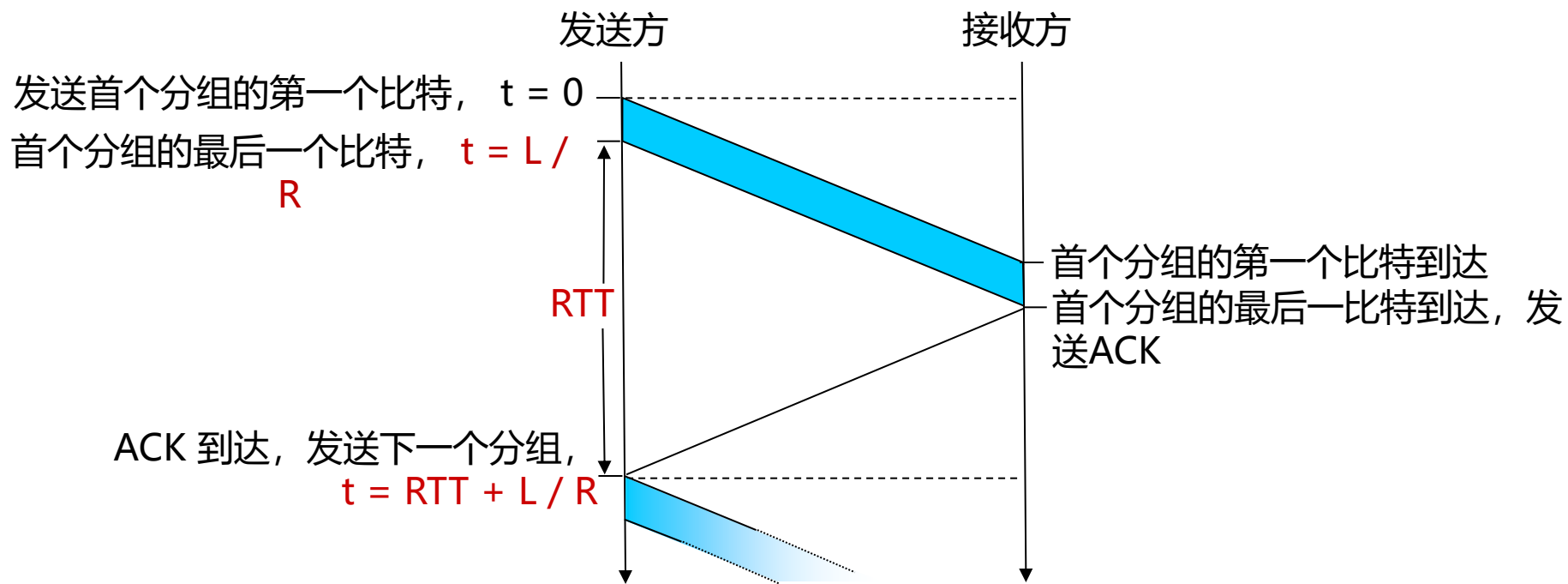
3.7 TCP拥塞控制

3.8 传输层功能的演化

rdt3.0 性能 (stop-and-wait)

- rdt3.0可以工作，但链路容量比较大的情况下，性能很差
 - 链路容量比较大，一次发一个分组，无法充分利用链路传输能力
 - U_{sender} : 利用率 – 忙于发送比特的时间比例
- 例：1 Gbps的链路，15 ms 端-端传播延时，分组大小为1kB (8000bit)

rdt3.0: 停-等操作

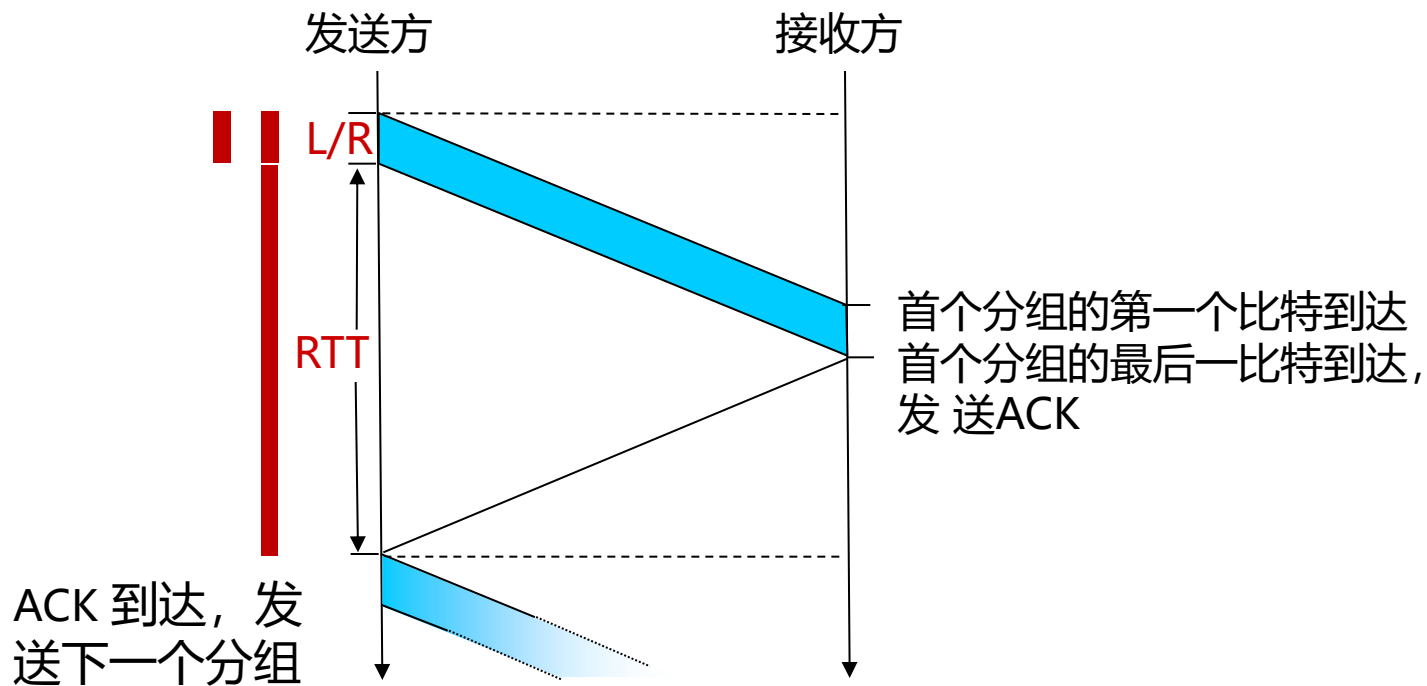


- 例: 1 Gbps的链路, 15 ms 端-端传播延时, 分组大小为1kB (8000bit)

rdt3.0: 停-等操作

- 例：1 Gbps的链路，15 ms 端-端传播延时，分组大小为1kB (8000bit)

$$\begin{aligned}U_{\text{sender}} &= \frac{L / R}{RTT + L / R} \\&= \frac{.008}{30.008} \\&= 0.00027\end{aligned}$$

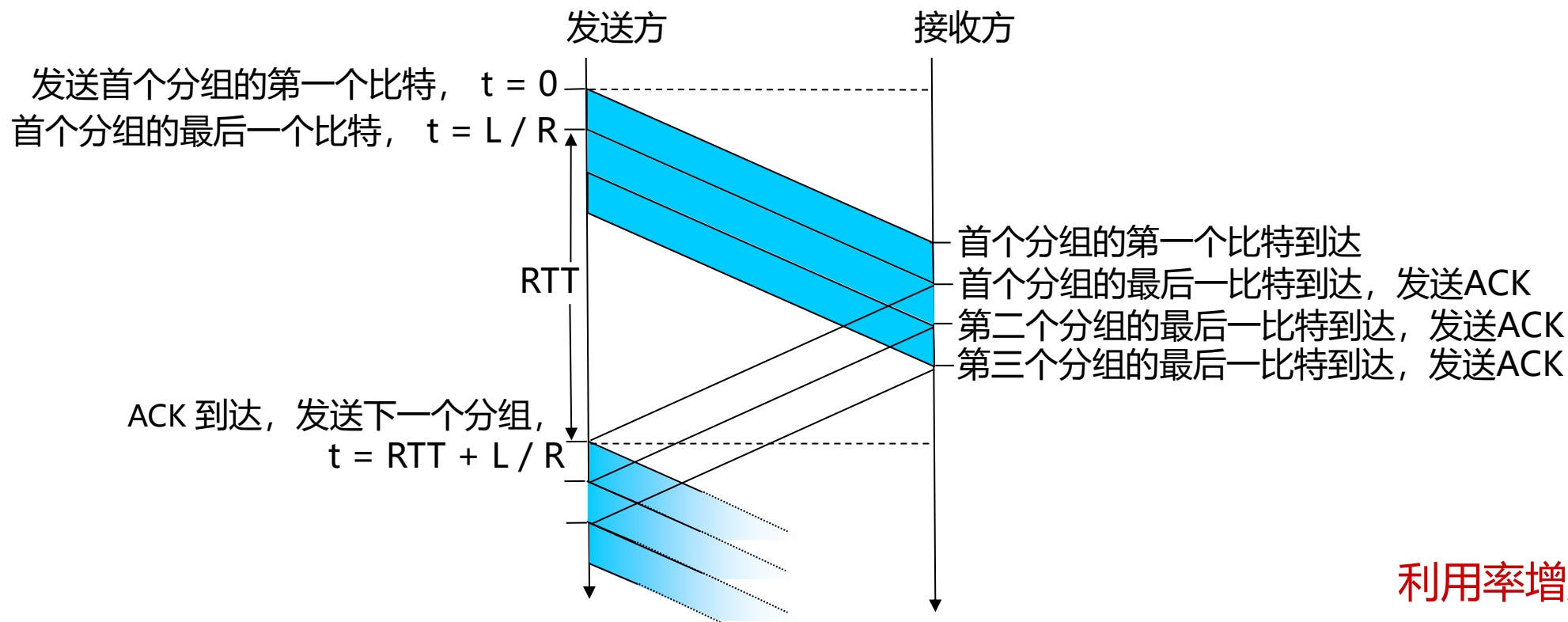


- 在 1Gbps 链路上只发送了 270kbps 的数据
- 瓶颈在于：网络协议限制了物理资源的利用！

$$T_{\text{transmit}} = \frac{L \text{ (分组长度, 比特)}}{R \text{ (传输速率, bps)}} = \frac{8\text{kb/pkt}}{10^9 \text{ b/sec}} = 8\mu\text{s}$$

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

流水线：提高链路利用率



- 增加 n , 能提高链路利用率
- 但当达到某个 n , 其 $u=100\%$ 时, 无法再通过增加 n , 提高利用率
- 瓶颈转移了 -> 链路带宽

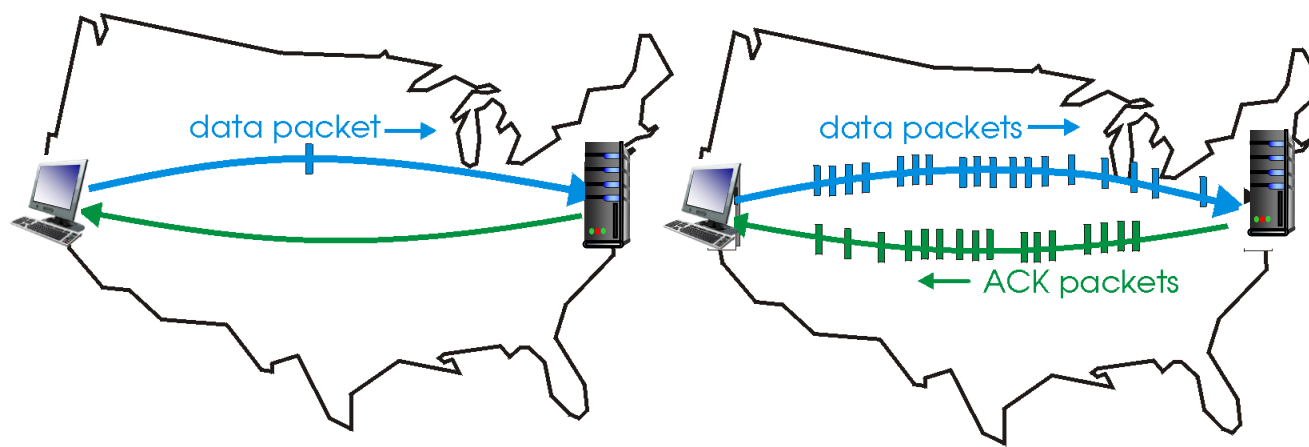
$$U_{\text{sender}} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

利用率增加到3倍!

rdt3.0: 流水线 (pipelining) 协议

流水线：允许发送方在**未得到对方确认**的情况下一次发送**多个分组**

- 必须增加序号的范围:用多个bit表示分组的序号
- 在发送方/接收方要有缓冲区
 - 发送方缓冲：未得到确认，可能需要重传；
 - 接收方缓存：上层用户取用数据的速率 $\neq$ 接收到的数据速率；接收到的数据可能乱序，排序交付（可靠）



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

两种通用的流水线协议

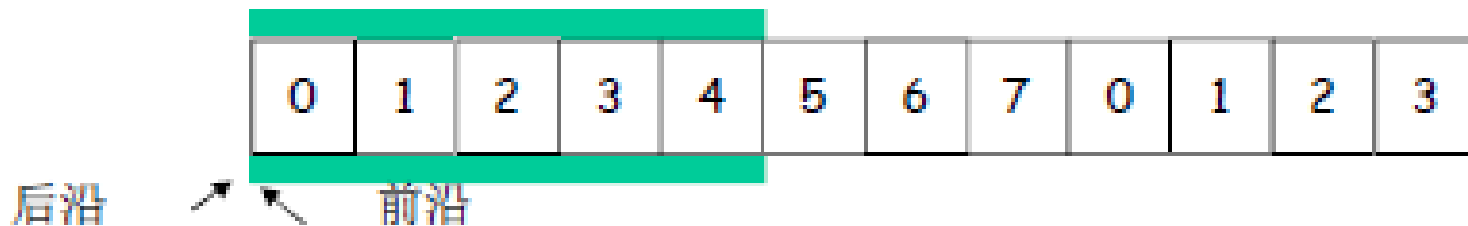
- 回退 N 步 Go-Back-N (GBN) , 滑动窗口协议
- 选择重传 Selective Repeat (SR)
- 逻辑：
 - 滑动窗口
 - 发送窗口
 - 接收窗口 (两种协议实现逻辑不同)

通用：滑动窗口（slide window）协议

- 发送缓冲区
 - 形式：内存中的一个区域，落入缓冲区的分组可以发送
 - 功能：用于存放已发送，但是没有得到确认的分组
 - 必要性：需要重发时可用
- 发送缓冲区大小：一次最多可以发送多少个未经确认的分组
 - 停止等待协议=1
 - 流水线协议 >1 ，合理的值，不能很大，链路利用率不能够超100%
- 发送缓冲区中的分组
 - **未发送的**：落入发送缓冲区的分组，可以连续发送出去；
 - **已经发送出去的、等待对方确认的分组**：发送缓冲区的分组只有得到确认才能删除

发送窗口

- 发送窗口：发送缓冲区内容的一个范围（发送缓冲区的子集）
 - 那些**已发送**但是**未经确认**的分组序号构成的空间
- 发送窗口的最大值 $\leq$ 发送缓冲区的值
- 一开始：没有发送任何一个分组
 - 后沿 (基序号, base) = 前沿 (下一个序号, nextSeqNum)
 - 之间为发送窗口的尺寸=0
- 每**发送**一个分组，**前沿**前移一个单位



发送缓冲区与发送窗口

- 发送缓冲区

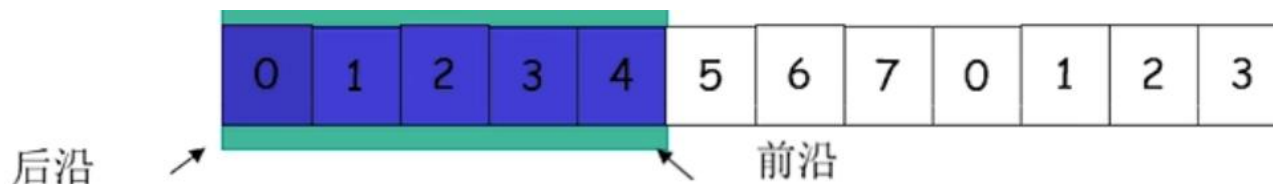
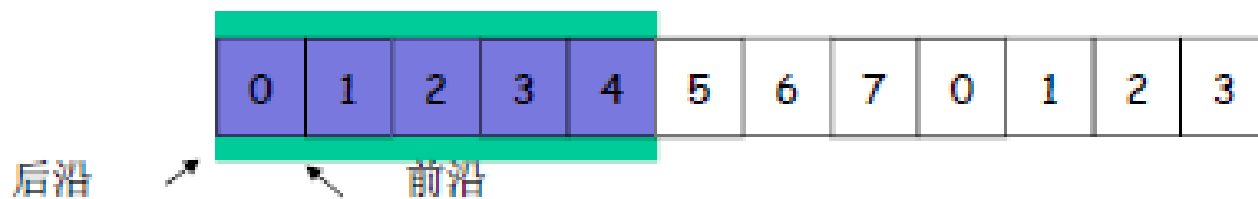
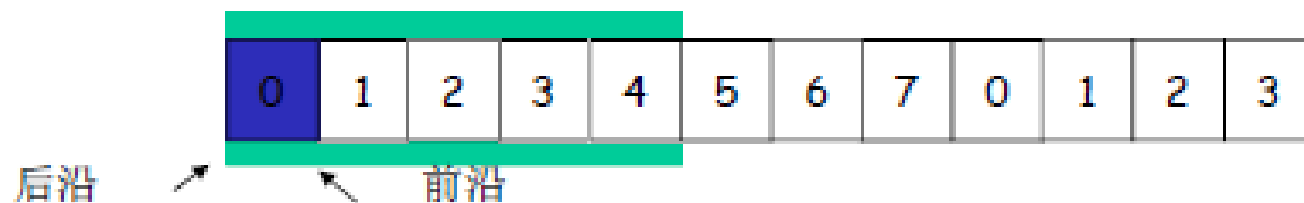
- 包含**已发送**分组和**未发送**分组
- 发送缓冲区大小 = 给定滑动窗口大小

- 发送窗口

- **发送缓冲区的子集**，前沿到后沿的部分(课本中：基序号到下一个序号部分)。
- 如果收到确认，则发送窗口的后沿要向前移动

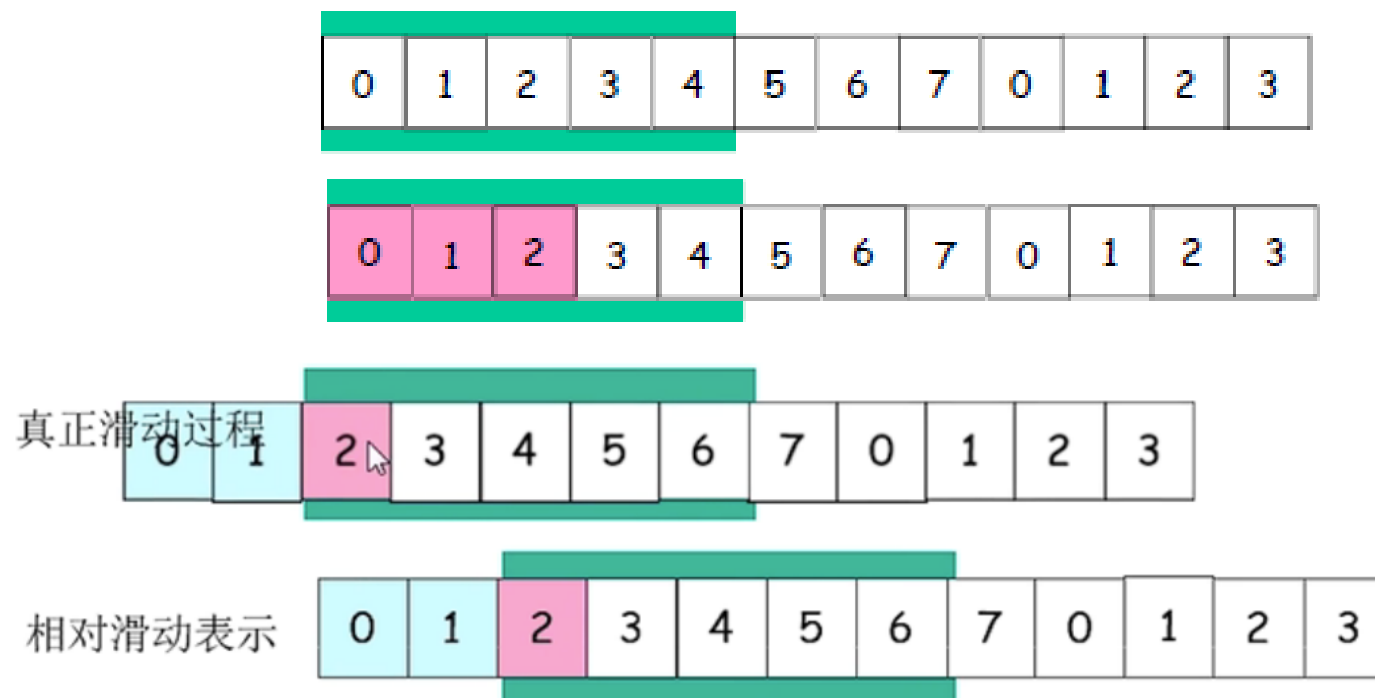
发送窗口的移动->前沿移动

- 发送窗口前沿移动的极限：不能够超过发送缓冲区



发送窗口滑动过程-相对表示方法

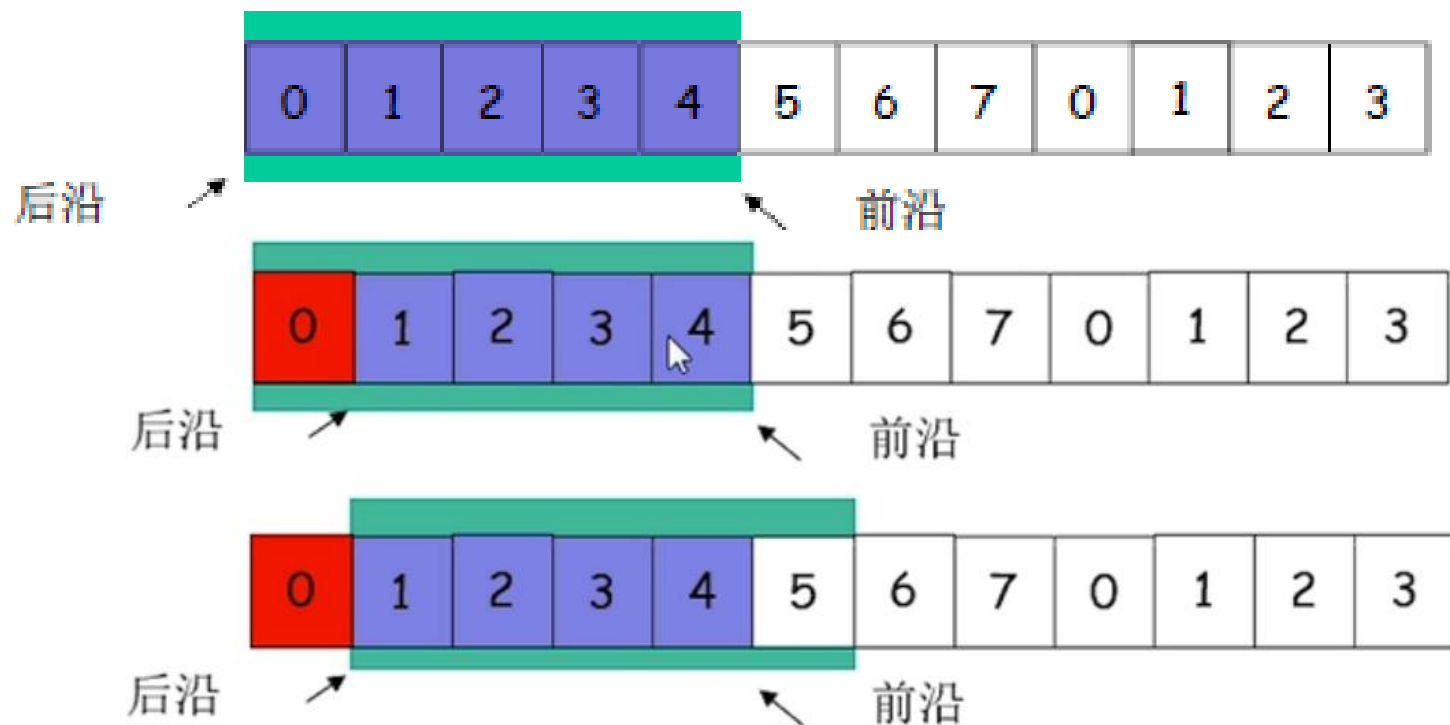
- 采用相对移动方式表示，分组不动
- 可缓冲范围移动，代表一段可以发送的内容



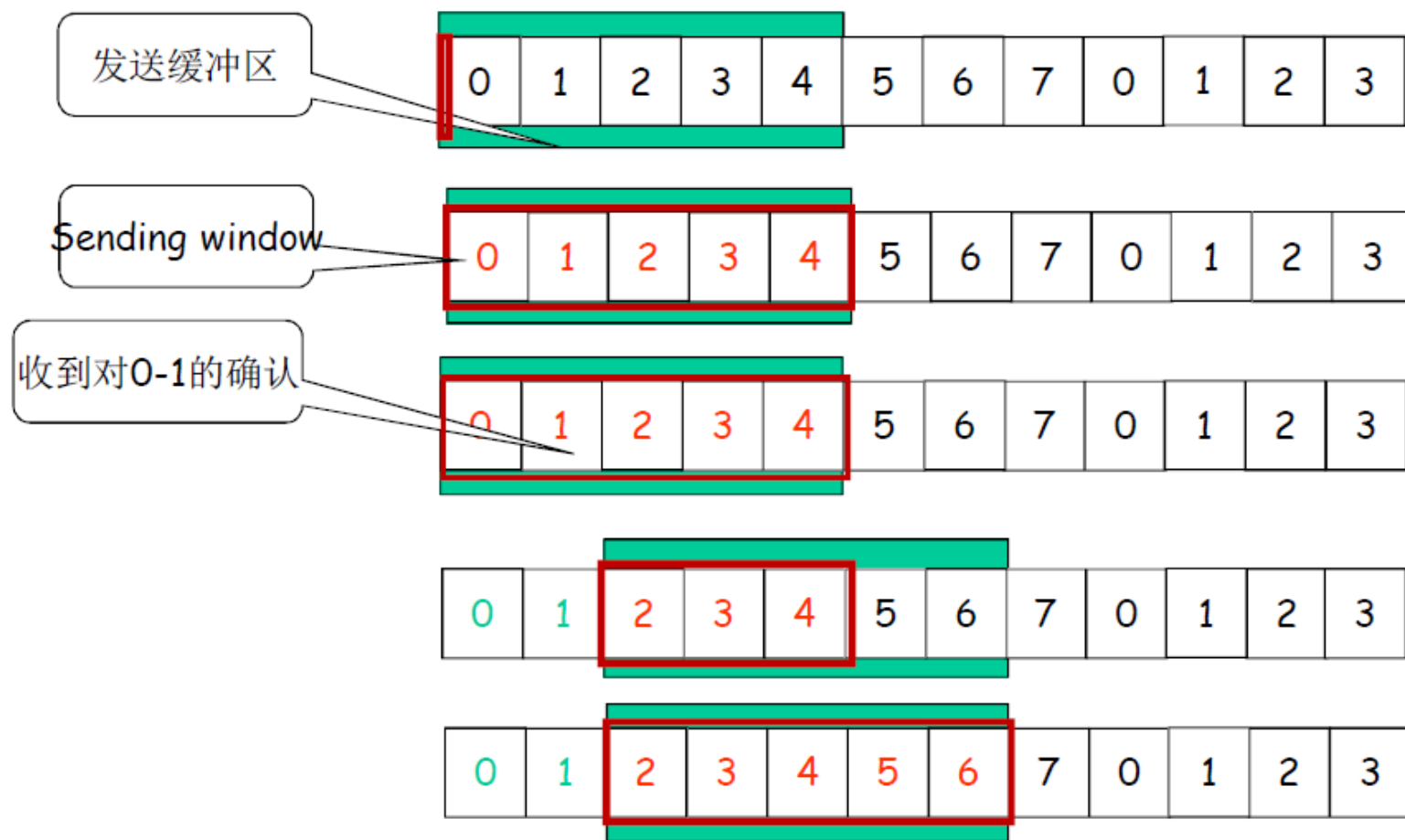
发送窗口的移动->后沿移动

■ 发送窗口后沿移动

- 条件：收到老分组的确认
- 结果：发送缓冲区罩住新的分组，来了分组可以发送
- 移动的极限：不能够超过前沿

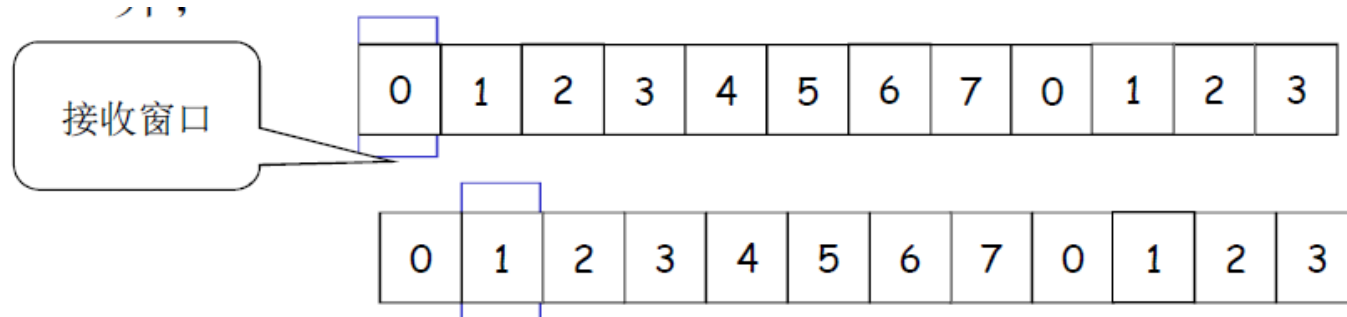


滑动窗口技术 —— 发送窗口



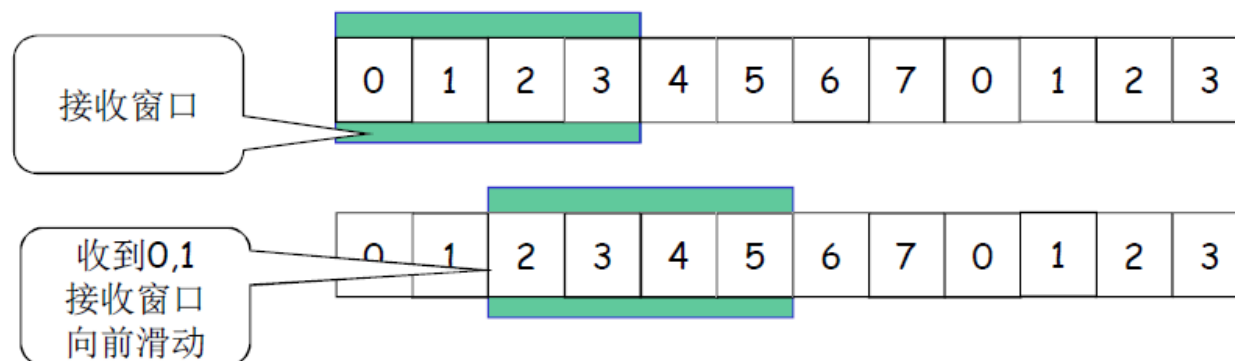
滑动窗口技术 —— 接收窗口

- 接收窗口 (receiving window) = 接收缓冲区
 - 接收窗口用于控制哪些分组可以接收；
 - 只有收到**接收窗口内的分组序号**才允许接收
 - 若序号在接收窗口之外，则丢弃；
 - GBN: 接收窗口尺寸 $W_r=1$ ，则**只能顺序**接收；
 - SR: 接收窗口尺寸 $W_r>1$ ，则**可以乱序**接收
 - 但提交给上层的分组，要按序
 - 例子： $W_r=1$ ，在 0 的位置；只有 0 号分组可以接收；
向前滑动一个，罩在 1 的位置，如果来了第 2/3 号分组，属于乱序分组，则丢弃；

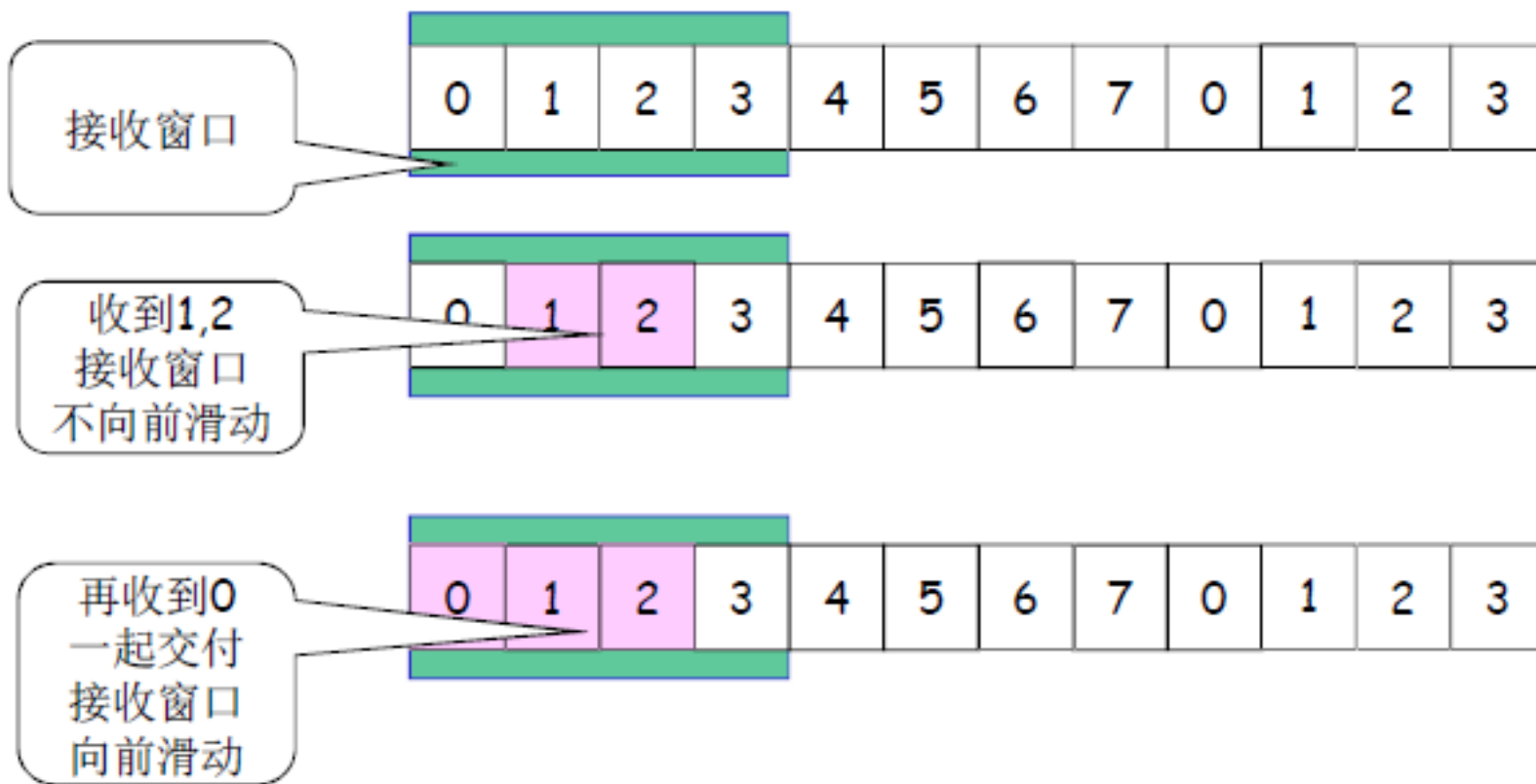


滑动窗口技术 —— 接收窗口

- 接收窗口的滑动和发送确认滑动：
 - 滑动：
 - 低序号的分组到来，接收窗口移动；
 - 高序号分组乱序到，缓存但不交付（因为要实现 rdt，不允许失序），不滑动
 - 发送确认：
 - GBN: 接收窗口尺寸 = 1； 发送连续收到的最大的分组确认（累计确认）
 - SR: 接收窗口尺寸 > 1； 收到哪个分组，就发送该分组的确认（非累计确认）



滑动窗口技术 -接收窗口 (SR)



正常情况下的2个窗口互动

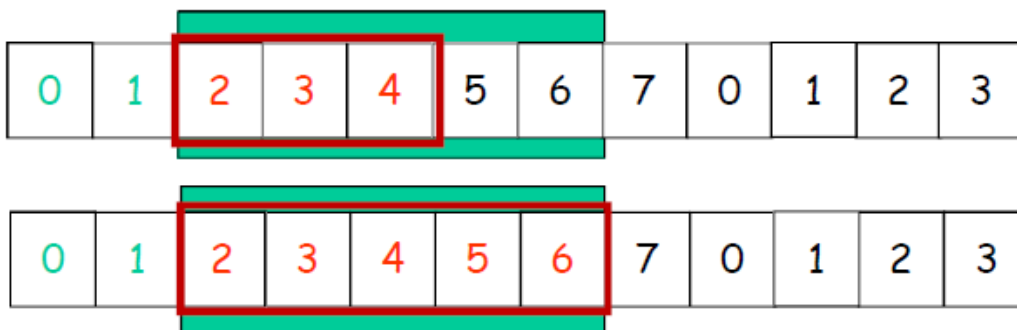
■ 发送窗口

- 有新的分组落入发送缓冲区范围，发送->前沿滑动
- 来了老的低序号分组的确认->后沿向前滑动->新的分组可以落入发送缓冲区的范围

■ 接收窗口

- 收到分组，落入到接收窗口范围内，接收
- 是低序号，发送确认给对方

- 发送端上面来了分组->发送窗口滑动->接收窗口滑动->发确认



异常情况下GBN的收发窗口互动

■ 发送窗口

- 新分组落入发送缓冲区范围，发送->前沿滑动
- 超时重发机制让发送端将发送窗口中的所有分组发送出去
- 来了**老分组的重复确认**->后沿不向前滑动->新的分组无法落入发送缓冲区的范围（此时如果发送缓冲区有新的分组可以发送）

■ 接收窗口

- 收到乱序分组，没有落入到接收窗口范围内，**抛弃**
- （重复）**发送老分组的确认，累计确认；**

异常情况下SR的收发窗口互动

■ 发送窗口

- 新分组落入发送缓冲区范围，发送->前沿滑动
- 超时重发机制让发送端将超时的分组重新发送出去
- 来了乱序分组的确认->后沿不向前滑动->新的分组无法落入发送缓冲区的范围
(此时如果发送缓冲区有新的分组可以发送)

■ 接收窗口

- 收到乱序分组，落入到接收窗口范围内，接收
- 发送该分组的确认，单独确认；

GBN协议和SR协议的异同

■ 相同之处

- 发送窗口 >1
- 一次能够可发送多个未经确认的分组

■ 不同之处

- GBN: 接收窗口尺寸 $=1$
 - 接收端: 只能顺序接收
 - 发送端: 从表现来看, 一旦一个分组没有发成功, 如: 0,1,2,3,4; 假如1未成功, 234都发送出去了, 要返回1再发送;
GB1
- SR: 接收窗口尺寸 >1
 - 接收端: 可以乱序接收
 - 发送端: 发送0,1,2,3,4, 一旦1未成功, 2,3,4,已发送, 无需重发, 选择性发送1

流水线 (pipelining) 协议：总结

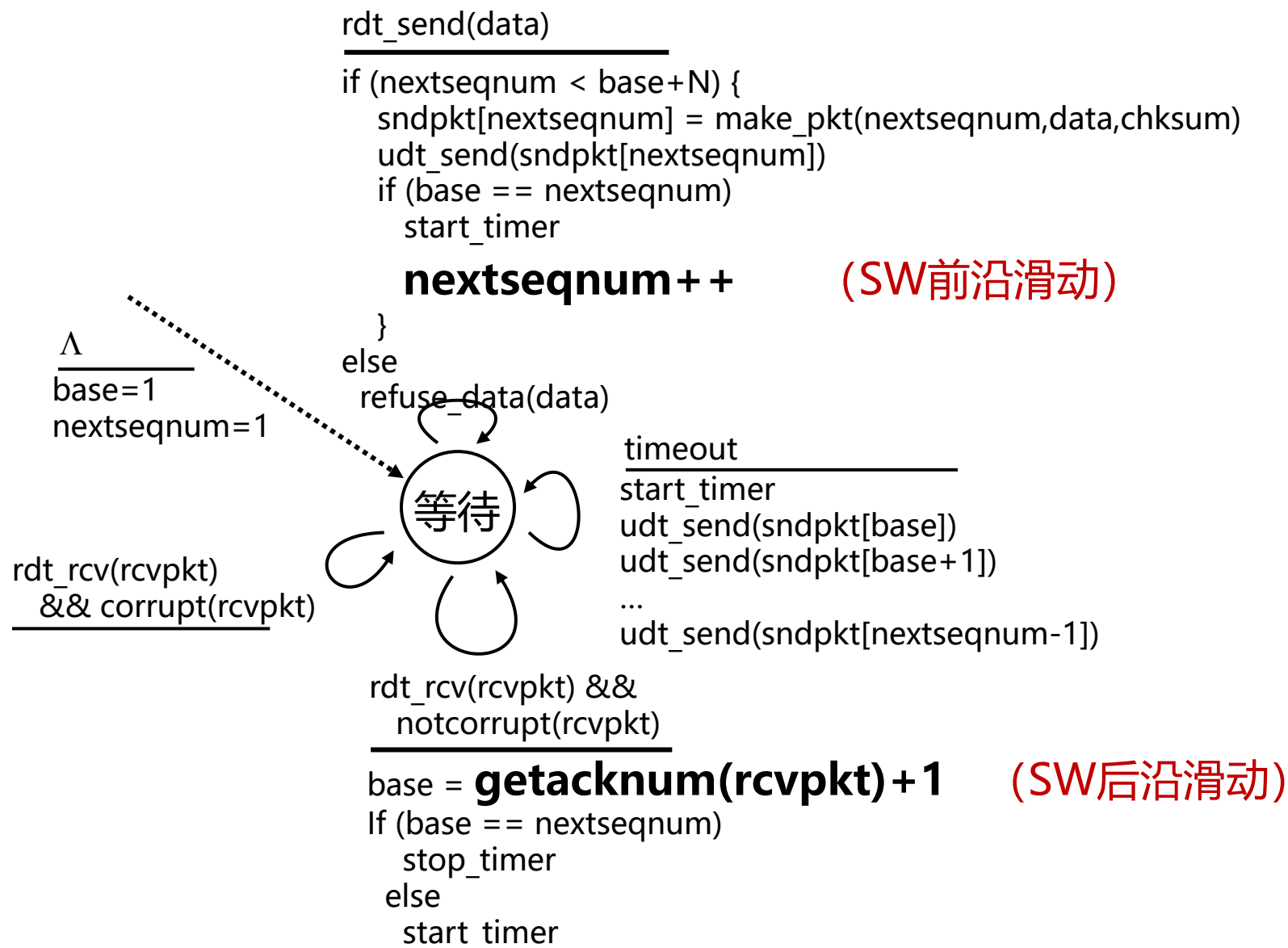
■ Go-back-N:

- 发送端最多在流水线中有N个未确认的分组
- 接收端只是发送累计型确认 **cumulative ack**
 - 接收端如果发现 **gap**，不确认新到来的分组
- 发送端拥有对最老的未确认分组的定时器
 - 只需设置**一个定时器**
 - 如果定时器到时，则**重传所有未确认分组**

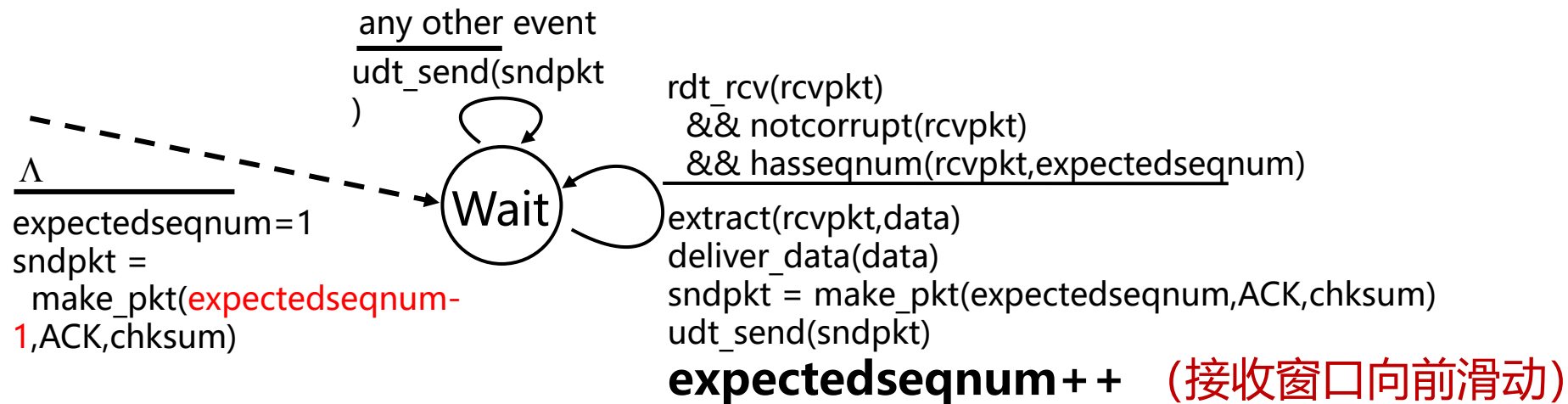
■ Selective Repeat:

- 发送端最多在流水线中有N个未确认的分组
- 接收方对每个到来的分组单独确认 **individual ack** (非累计确认)
- 发送方为每个**未确认的分组**保持一个**定时器**
 - 当超时定时器到时，只是重发到时的未确认分组

Go-Back-N: 发送方扩展的FSM



Go-Back-N: 接收方扩展的FSM



- 只发送ACK:对顺序接收的最高序号的分组
 - 可能会产生重复的ACK
 - 只需记住 expectedseqnum; 接收窗口 = 1
 - 只一个变量就可表示接收窗口
- 对乱序的分组:
 - 丢弃 (不缓存) → 在接收方不会被缓存!
 - 对顺序接收的最高序号的分组进行确认-累计确认

运行中的 GBN

sender window (N=4)

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

在接收端，乱序的
不缓存；
因此哪个n分组丢失，
即GB到那个分组n；
即使n以后的分组传
送都是正确的

sender

send pkt0
send pkt1
send pkt2
send pkt3
(wait)

rcv ack0, send pkt4
rcv ack1, send pkt5

ignore duplicate ACK



pkt 2 timeout

send pkt2
send pkt3
send pkt4
send pkt5

receiver

receive pkt0, send ack0
receive pkt1, send ack1

receive pkt3, discard,
(re)send ack1

receive pkt4, discard,
(re)send ack1

receive pkt5, discard,
(re)send ack1

rcv pkt2, deliver, send ack2
rcv pkt3, deliver, send ack3
rcv pkt4, deliver, send ack4
rcv pkt5, deliver, send ack5

选择重传 Selective repeat (SR)

- 接收方对每个正确接收的分组，分别发送 ACKn（非累积确认）
 - 接收窗口 > 1
 - 可以缓存乱序的分组
 - 最终将分组按顺序交付给上层
- 发送方只对那些没有收到ACK的分组进行重发-选择性重发
 - 发送方为每个未确认的分组设定一个定时器
- 发送窗口的最大值（发送缓冲区）限制发送未确认分组的个数

选择重传: 发送方和接收方

发送方

从上层接收数据:

- 如果下一个可用于该分组的序号可在发送窗口中, 则发送

timeout(n):

- 重新发送分组n, 重新设定定时器

ACK(n) in

[sendbase, sendbase+N]:

- 将分组n标记为已接收
- 如n为最小未确认的分组序号, 将base移到下一个未确认序号

接收方

分组 n [rcvbase, rcvbase+N-1]

- 发送 ACK(n)
- 乱序: 缓存
- 有序: 该分组及以前缓存的序号连续的分组交付给上层, 然后将窗口移到下一个仍未被接收的分组

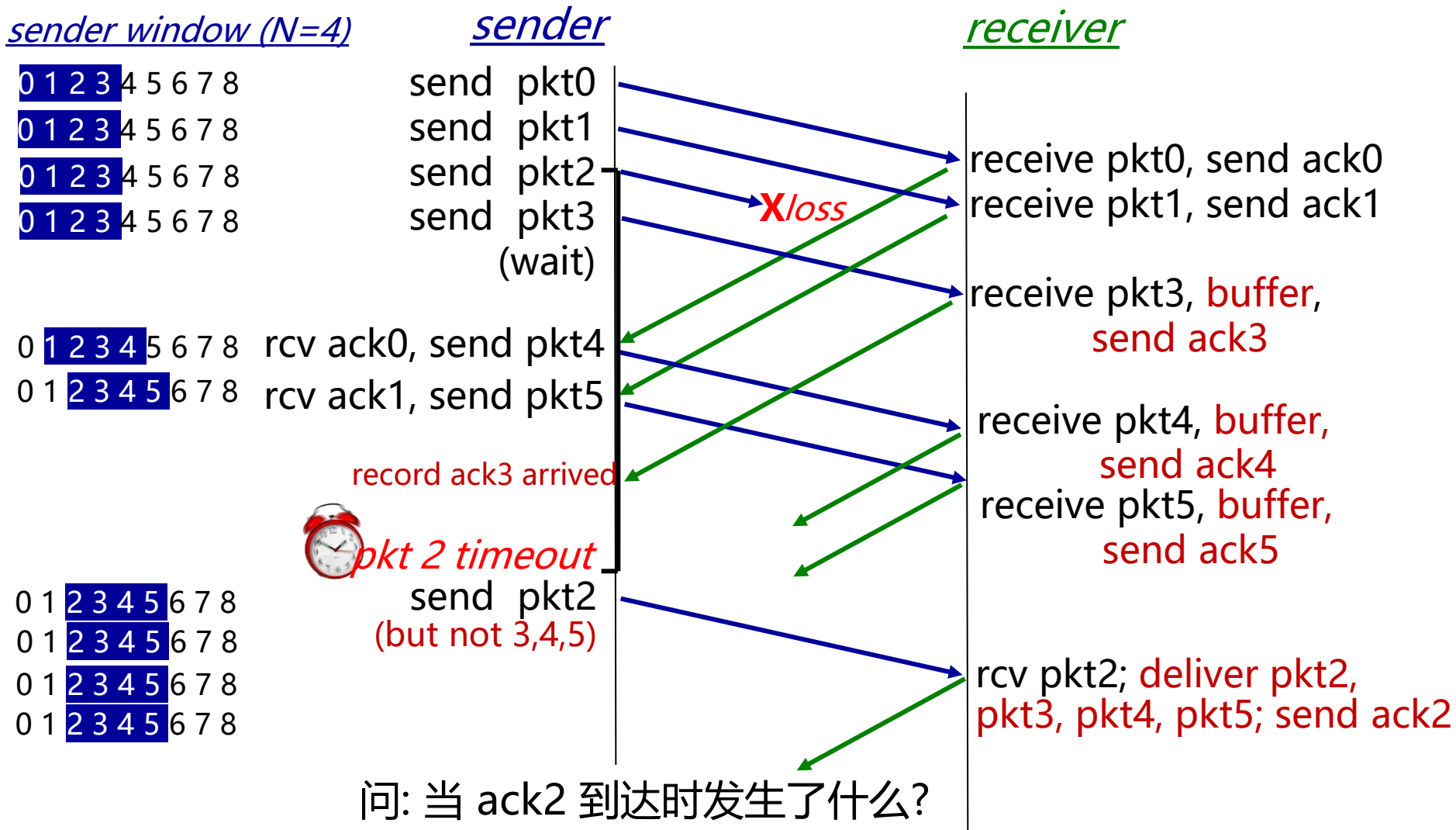
分组 n [rcvbase-N, rcvbase-1]

- ACK(n)

其它:

- 忽略该分组

选择重传 SR 的运行



对比GBN和SR

	GBN	SR
优点	简单，所需资源少(接收方一个缓存单元)	出错时，重传一个代价小
缺点	一旦出错，回退N步代价大	复杂，所需要资源多(接收方多个缓存单元)

■ 适用范围

- 出错率低：比较适合GBN，出错非常罕见，没有必要用复杂的SR，为罕见的事件做日常的准备和复杂处理
- 链路容量大（延迟大、带宽大）：比较适合SR而不是GBN，一点出错代价太大

SR接收窗口困境

■ 发送窗口限制（最大值）

- GBN: $2^n - 1$ （ n 为序号使用的比特位数）
- SR: 2^{n-1}

例如: $n=2$; 序列号: 0, 1, 2, 3

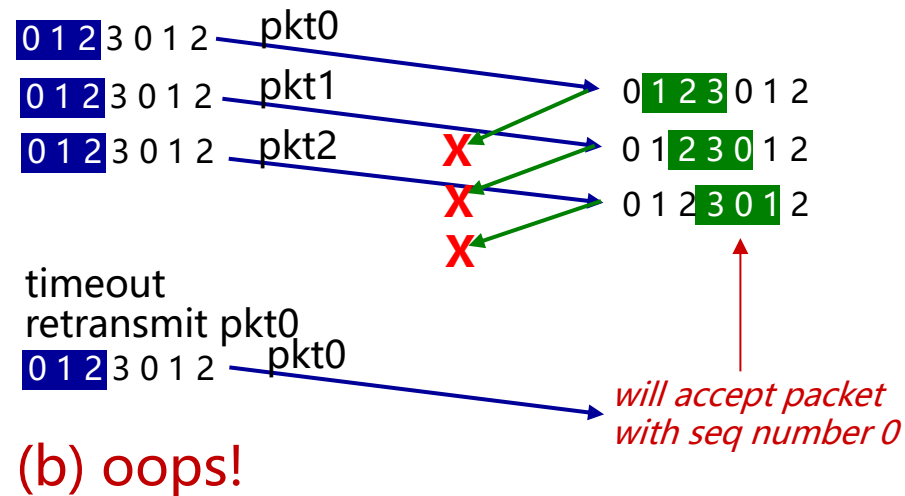
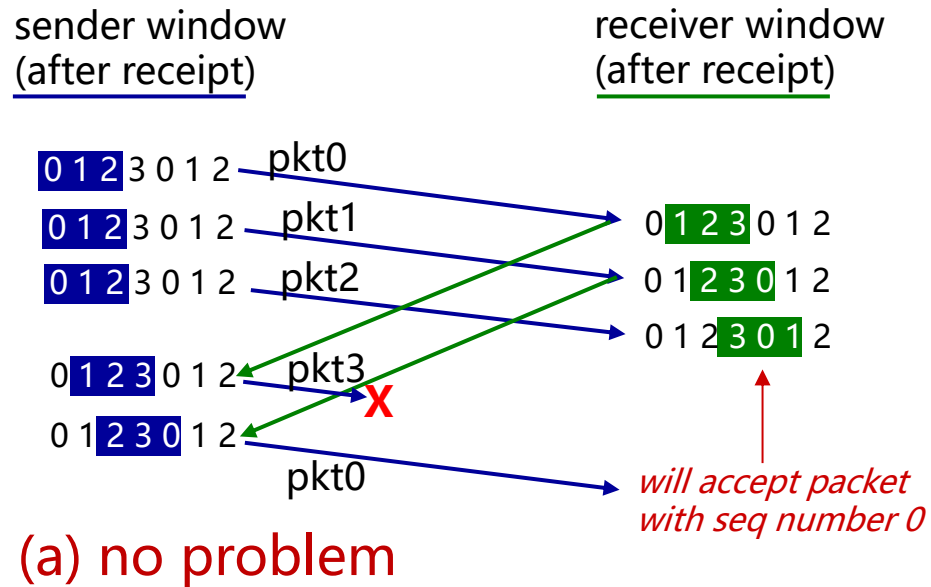
- GBN = 3
- $R=2$

SR的例子:

- 接收方看不到二者的区别!
- 将重复数据误认为新数据 (a)

Q: 序号大小与发送窗口大小之间的关系?

A: SR的发送窗口须小于等于分组标号长度的一半



第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化